Computer Vision Final project

Task 1: Selective Object Enhancement

Objective

Enhance a specific-colored object (sharpen) while blurring the rest of the image.

Methodology

Pipeline:

text

Input → HSV Conversion → Color Mask → Noise Removal → Sharpen (mask) + Blur (background) → Output

1.1 Blur

- Box blur using a normalized kernel of size k×k
- Each pixel becomes the average of its k×k neighborhood
- Implemented with sliding_window_view for vectorized computation

1.2 Sharpen

- Sobel edge detection (Gx and Gy kernels)
- Sharpened = Original + strength × (Gx + Gy)
- Strength parameter controls intensity

1.3 Color Mask

- Convert RGB to HSV (better color separation)
- Create binary mask using HSV bounds for target color
- Red requires two ranges (0-10 and 170-180)
- Remove small noise by filtering contours below min_area

Results

Color	Target Object	Image
Red	Roses	
Yellow	Sunflowers	
Green	Stems	

Task 2: Comparative Study of Sharpening Pipelines for X-Ray Images

Objective

Design and implement three different image enhancement pipelines for X-ray images, each using a different strategy. Compare the results and analyze the strengths and weaknesses of each approach.

Library Design

All Task 2 code is organized in a single module: task_2.py. The file is structured in four layers:

- **Shared I/O utilities** — load_image and save_image handle reading and writing grayscale images using OpenCV.
- **Shared processing utilities** — apply_kernel performs manual 2D convolution from scratch using NumPy stride tricks (no cv2.filter2D). clip_and_convert ensures all float results are clipped to [0,255] and converted to uint8.
- **Three independent pipelines** — each pipeline is a self-contained function that takes a grayscale image and returns an enhanced image. Helper functions (e.g. _gaussian_kernel, _apply_clahe) are prefixed with underscore to indicate they are internal.
- **compare_pipelines runner** — a utility function that runs all three pipelines on the same image and saves a side-by-side comparison strip.

Module structure:

```
task_2.py
■■■ load_image(path)
■■■ save_image(image, path)
■■■ apply_kernel(image, kernel) # shared convolution
■■■ clip_and_convert(image) # shared utility
■■■ _gaussian_kernel(size, sigma) # P1 helper
■■■ pipeline_unsharp_masking(...) # Pipeline 1
■■■ _apply_clahe(...) # P2 helper
■■■ pipeline_clahe_laplacian(...) # Pipeline 2
■■■ pipeline_fft_highpass(...) # Pipeline 3
```

Methodology

One chest X-ray image is used as input for all three pipelines to ensure a fair comparison — any difference in output is caused only by the pipeline, not by the image content.

```
Input  Pipeline 1 / Pipeline 2 / Pipeline 3  Side-by-side Comparison
```

1.1 Pipeline 1 — Unsharp Masking

Concept:

Blur the image with a Gaussian filter to get a smooth version with no edges, subtract the blurred from the original to isolate edges and details (the unsharp mask), then add the mask back scaled by a strength parameter to amplify those edges.

Formula:

```
mask = original - Gaussian_blurred
output = original + strength x mask
```

Steps:

- Build a 2D Gaussian kernel (size=5, sigma=1.0) — normalized so all values sum to 1.
- Convolve the image with the kernel using apply_kernel to get the blurred version.
- Subtract blurred from original to extract only edges and fine details.
- Add the edge mask back to the original scaled by strength=1.5.
- Clip to [0,255] and convert to uint8.

1.2 Pipeline 2 — CLAHE + Laplacian Sharpening

Concept:

X-ray images often have low contrast with uneven exposure. CLAHE fixes contrast region by region, then Laplacian sharpening makes the edges crisp.

Stage A — CLAHE:

- Divide the image into 64 tiles (8x8 grid). Each tile is an independent region.
- For each tile: build a histogram, clip it at clip_limit=2.0 to prevent noise amplification, redistribute excess pixels evenly.
- Build a CDF from the clipped histogram and normalize to [0,255] to create a LUT — a translation table that remaps pixel intensities to improve contrast.
- For every pixel, blend the LUTs of the 4 surrounding tile centers using bilinear interpolation to ensure smooth transitions with no harsh tile borders.

Stage B — Laplacian Sharpening:

Apply the Laplacian kernel to the CLAHE result to detect edges, then add them back:

```
[ 0, -1, 0 ] output = CLAHE_result + strength x Laplacian
[-1, 4, -1 ]
[ 0, -1, 0 ]
```

1.3 Pipeline 3 — FFT Gaussian High-Pass Filter

Concept:

Convert the image to the frequency domain. Low frequencies = smooth regions, high frequencies = edges and details. Boost high frequencies then convert back to pixels.

Steps:

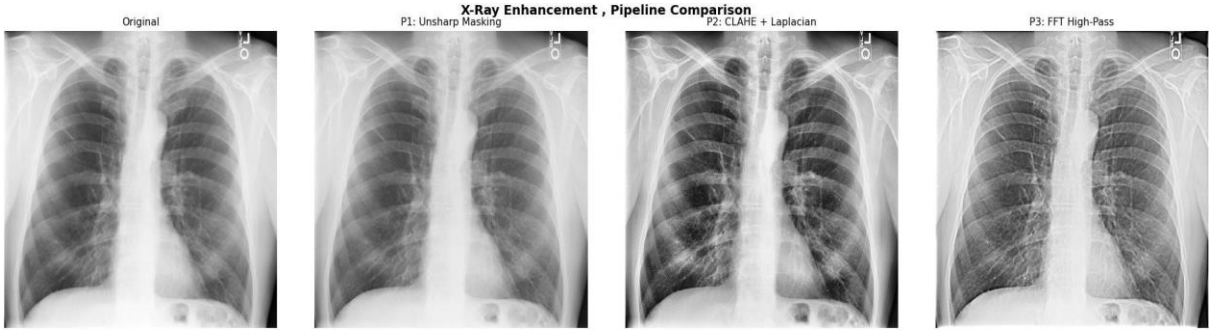
- Apply 2D FFT and shift DC (zero-frequency) to the center of the spectrum.
- Build a Gaussian High-Pass Filter: HPF = 1 - exp(-D^2 / (2 x cutoff^2)). Values near center (low freq) ~0, values far from center (high freq) ~1.
- Blend as: combined_filter = 1 + boost x HPF — keeps low frequencies but amplifies high ones.
- Multiply FFT by the filter, apply inverse shift, apply inverse FFT to return to pixels.
- Take magnitude (FFT produces complex numbers), clip to [0,255] and convert to uint8.

Strengths and Weaknesses

Pipeline	Strengths	Weaknesses
P1: Unsharp Masking	• Simple and efficient • Easy to tune • Good for well-exposed X-rays • Preserves brightness	• Cannot fix low contrast • Amplifies noise at high strength • Global — not adaptive
P2: CLAHE + Laplacian	• Best for dark/low-contrast X-rays • Adapts per-tile to uneven exposure • Powerful contrast + sharpening • Best for medical imaging	Artifacts if clip_limit too high • More parameters to tune • Laplacian may amplify noise
P3: FFT High-Pass	• Precise frequency-band control • Reveals fine textures • Mathematically principled • Independent boost and cutoff control	• Ringing artifacts near edges • Less intuitive parameters • Slightly slower on large images

Results and Observations

Output — Pipeline Comparison:



Pipeline 1 — Unsharp Masking:

The output shows a mild but visible improvement in sharpness. Rib edges and lung tissue boundaries appear slightly more defined compared to the original. The overall brightness and tone are preserved. However, the dark lung regions remain similarly dark — the pipeline sharpens what is already visible but does not reveal hidden detail in underexposed areas.

Pipeline 2 — CLAHE + Laplacian:

This pipeline produces the most dramatic transformation. The lung fields become much darker and higher in contrast, revealing internal vascular and bronchial structures that are barely visible in the original. Rib boundaries and soft tissue edges are sharp and well-defined. The adaptive per-tile contrast enhancement successfully handles the uneven exposure between the bright bone regions and the dark lung fields simultaneously. This result is the most diagnostically useful of the three.

Pipeline 3 — FFT High-Pass:

The result shows noticeably enhanced edge definition and fine texture detail. Rib edges and lung tissue patterns are sharper than the original and Pipeline 1, and the overall image appears slightly brighter due to the high-frequency boost. The enhancement is stronger than Pipeline 1 but does not achieve the contrast improvement of Pipeline 2. No visible ringing artifacts are present at the default boost=2.0 setting.

Overall Conclusion:

For general well-exposed X-rays, Pipeline 1 is sufficient and the simplest to apply. For dark, underexposed, or low-contrast X-rays — the common case in medical imaging — Pipeline 2 gives the best results by combining adaptive contrast enhancement with edge sharpening. Pipeline 3 provides a strong middle ground with fine texture enhancement using a mathematically principled frequency-domain approach.

Task 3: Intelligent Auto Image Enhancement System

Objective

The goal of this task is to develop an intelligent image enhancement system capable of automatically analyzing an input image and determining the most suitable enhancement method based on the image characteristics.

The system is designed to handle three common image quality problems:

- Dark / underexposed images
- Overexposed (very bright) images
- Low contrast images

After analyzing the image, the system automatically applies an enhancement technique to improve the visual quality of the image.

System Design

The implemented system follows a modular image processing pipeline consisting of four main stages:

1. Image Loading
2. Image Analysis
3. Image Classification
4. Automatic Enhancement

The implementation was developed from scratch using Python, NumPy, and OpenCV without relying on built-in enhancement functions.

Image Analysis Stage

To analyze the image characteristics, the RGB image is first converted into a grayscale representation.
The grayscale image is only used for analysis purposes, while enhancement operations are applied to the original image.

Two important statistical measures are calculated:

1. Mean Intensity

The mean intensity measures the overall brightness of the image.

- Low mean → dark image
- High mean → bright image

2. Standard Deviation

The standard deviation measures the contrast distribution in the image.

- Low standard deviation → low contrast image
- High standard deviation → image contains sufficient contrast

Image Classification

Based on the calculated statistics, the system automatically classifies the image into one of the following categories:

Category	Condition
Dark Image	Mean intensity < 80
Bright Image	Mean intensity > 180
Low Contrast Image	Standard deviation < 40
Normal Image	Otherwise

This decision-making stage allows the system to behave intelligently and select the proper enhancement technique automatically.

Enhancement Techniques

Different enhancement techniques were implemented for each image category.

1. Dark Image Enhancement

For dark images, Histogram Equalization was implemented from scratch.

Concept

Histogram Equalization redistributes pixel intensities over the full intensity range to improve brightness and reveal hidden details in dark regions.

Result

- Increased brightness
- Better visibility of objects and textures
- Improved overall image clarity

2. Bright Image Enhancement

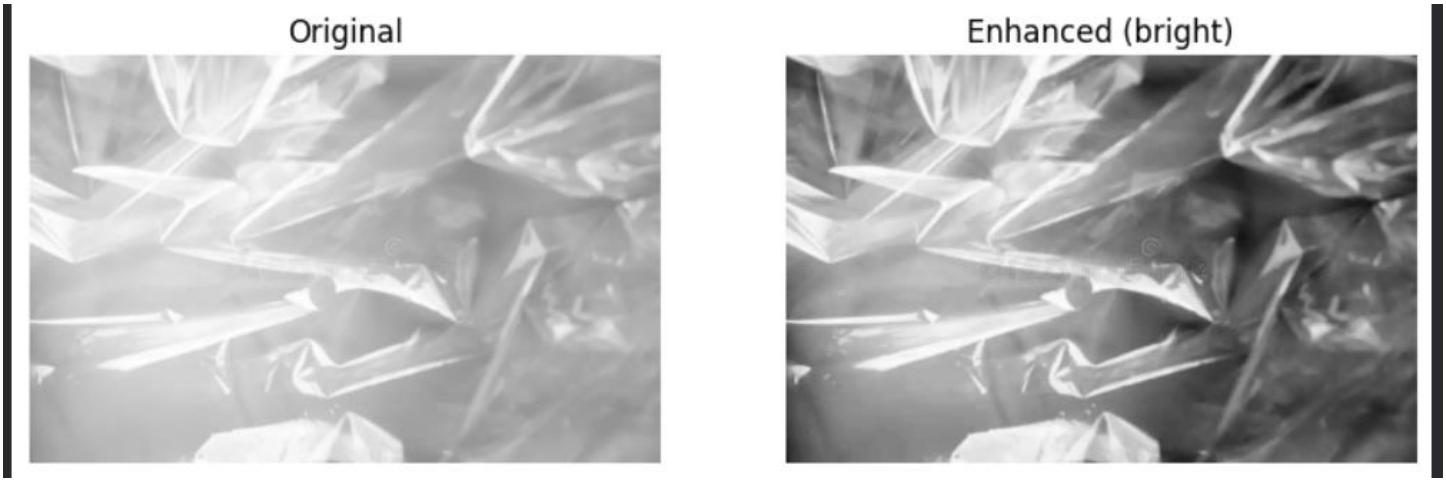
For overexposed images, Gamma Correction was applied.

Concept

Gamma correction adjusts image intensity using a nonlinear transformation.
This helps reduce excessive brightness while preserving image details.

Result

- Reduced overexposure
- More balanced illumination
- Improved visual comfort



3. Low Contrast Enhancement

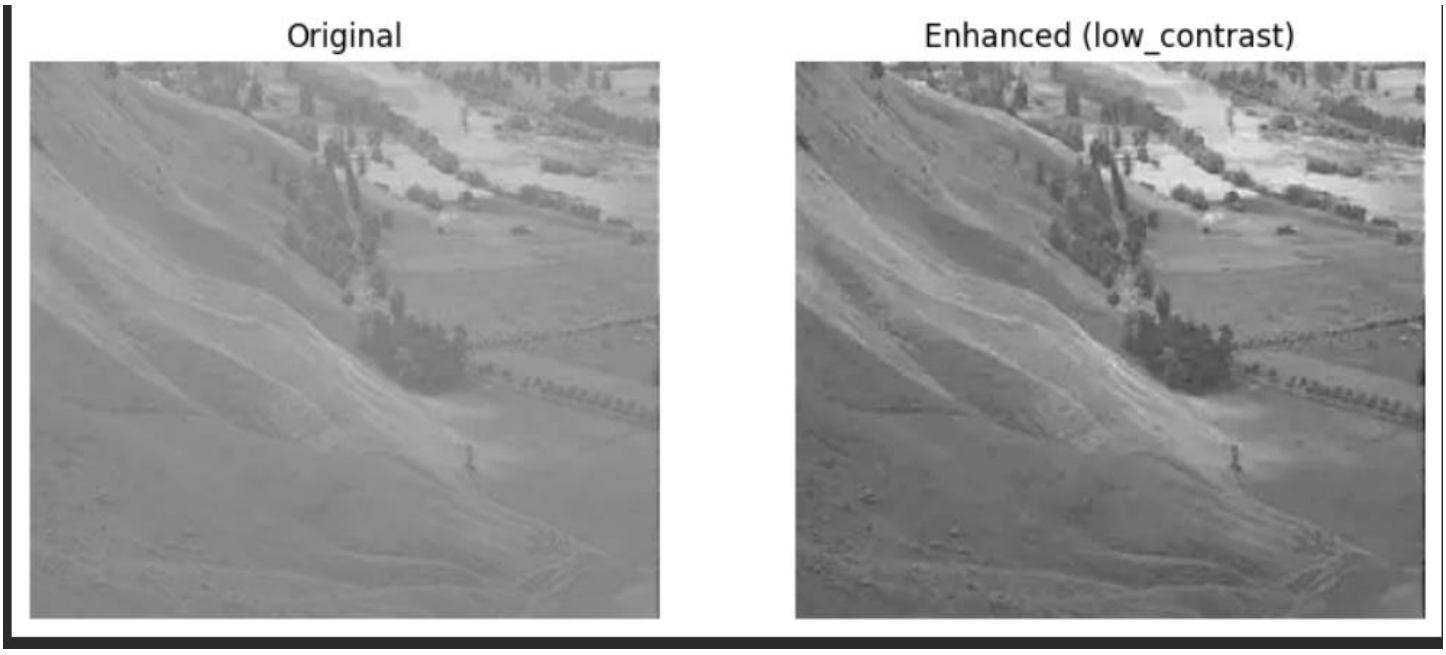
For low contrast images, Contrast Stretching was implemented.

Concept

Contrast stretching expands the intensity range between the darkest and brightest pixels.

Result

- Increased contrast
- Better separation between objects and background
- Improved visibility in foggy or faded images



Output Visualization

For each test image, the system displays:

- Original image
- Enhanced image
- Detected image category

The enhancement results demonstrate that the system successfully adapts its processing strategy according to the input image characteristics.

Experimental Results

Dark Image

The system successfully detected the image as dark and applied histogram equalization.
The output image became brighter and more visually clear compared to the original image.

Bright Image

The system classified the image as bright and applied gamma correction.
The excessive brightness was reduced while maintaining the original colors of the image.

Low Contrast Image

The system identified the image as low contrast and applied contrast stretching.
The resulting image showed clearer details and stronger contrast between image regions.

Conclusion

The developed intelligent enhancement system successfully performs automatic image analysis and adaptive enhancement.

The project demonstrates several important image processing concepts including:

- Image statistics analysis
- Automatic decision-making
- Histogram processing
- Gamma correction
- Contrast enhancement
- Modular image processing pipeline design

The system provides an effective and reusable framework for automatic image enhancement applications.

Task 4: Document Cleaning System

Objective

The goal of this task is to enhance scanned document images affected by noise, shadows, and uneven illumination, and produce a clean binary image where the text is clearly separated from the background.

Methodology

Pipeline:

Input → Grayscale → Background Estimation → Background Removal → Normalization → Inversion → Thresholding → Output

1. Grayscale Conversion

The input RGB image is converted to grayscale using a weighted sum of channels:

- Red, Green, and Blue channels are combined using perceptual weights.
- This simplifies the image while preserving intensity structure needed for document analysis.

2. Background Estimation (Median Filtering)

A large kernel median filter (21×21) is applied to approximate the document background:

- The median filter smooths out text and retains only large-scale illumination patterns.
- This estimated background represents shadows and lighting variations.

3. Background Removal

The estimated background is subtracted from the grayscale image:

background - grayscale

- This step isolates foreground text from illumination artifacts.
- Intensity clipping is applied to keep values within [0,255].

4. Normalization (Contrast Stretching)

The resulting image is normalized to improve visibility:

- Pixel values are rescaled based on min and max intensity.
- This enhances contrast between text and background.

5. Inversion

The image is inverted so that:

- Text becomes dark
- Background becomes white

This matches standard document format expectations.

6. Thresholding

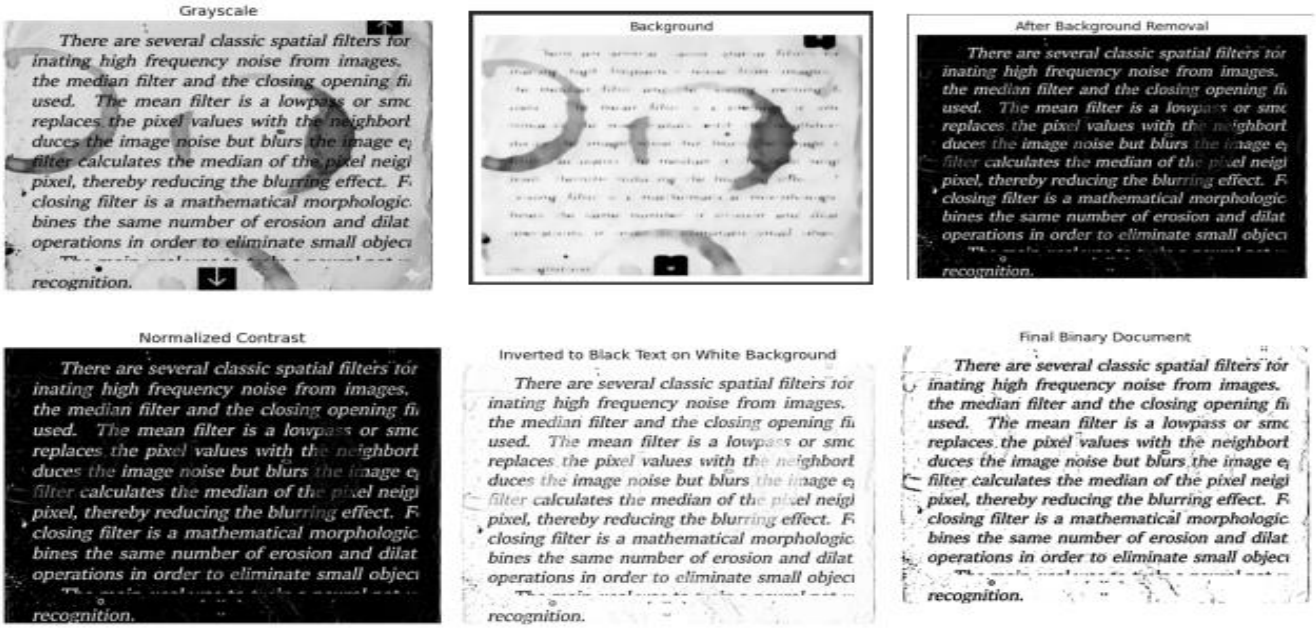
A global threshold is applied using the mean intensity of the image:

- Pixels above threshold → white (background)
- Pixels below threshold → black (text)

This produces the final binary document.

Strengths and Weaknesses

Stage	Strengths	Weaknesses
Median Background Estimation	Simple and effective for Ilumination removal	May blur fine text details
Background Subtraction	Enhances text visibility	Sensitive to extreme lighting conditions
Normalization	Improves contrast significantly	Can amplify noise if image is very degraded
Global Thresholding	Simple and fast	Not adaptive to local variations
Full Pipeline	Fully from scratch, interpretable, effective	Not as robust as adaptive thresholding methods



Results and Observations

The pipeline significantly improves document readability by removing uneven illumination and enhancing text contrast.

- Shadows that previously obscured text regions are successfully eliminated.
- Background becomes uniform and white after inversion.
- Text becomes clearly distinguishable in binary form.

However, in cases of severe degradation or very faint text, global thresholding may fail to preserve all characters, suggesting that adaptive thresholding could further improve robustness.

Overall Conclusion

The implemented pipeline demonstrates a complete from-scratch document enhancement system that effectively handles noise and illumination problems using classical image processing techniques.

It provides a strong baseline for document cleaning tasks and clearly shows the impact of each processing stage in improving readability.

Task 5: Panorama Stitching System

Objective

The goal of this task is to create a panorama image by stitching two overlapping images of the same scene. The system detects matching keypoints between the images, estimates a geometric transformation, and warps one image into the coordinate space of the other to form a single wide view.

Methodology

Pipeline

Input Images → Feature Detection → Feature Matching → Outlier Removal (RANSAC) → Homography Estimation → Image Warping → Stit

1. Feature Detection (SIFT)

Scale-Invariant Feature Transform (SIFT) is used to detect keypoints and extract descriptors from both images.

- Keypoints represent distinctive image regions (edges, corners, textures).
- Descriptors encode local neighborhood information around each keypoint.
- SIFT is robust to scale, rotation, and illumination changes.

2. Feature Matching (Brute Force + Ratio Test)

- A Brute Force matcher compares descriptors between both images.
- For each descriptor, the two nearest neighbors are found.

- Lowe’s ratio test is applied:

good match if d1<0.7×d2\text{good match if } d_1 < 0.7 \times d_2good match if d1<0.7×d2

This removes ambiguous or weak matches and keeps only reliable correspondences.

3. Outlier Removal (RANSAC)

- Even after matching, some correspondences are incorrect.
- RANSAC (Random Sample Consensus) is used to remove outliers.
- It estimates the best transformation while ignoring mismatched points.

4. Homography Estimation

A homography matrix HHH is computed:

x′=Hxx' = Hxx′=Hx

- It maps points from the second image into the coordinate system of the first image.
- Computed using RANSAC-validated correspondences.

5. Image Warping and Stitching

- The second image is warped using the homography matrix.
- Canvas size is dynamically calculated to fit both images.
- A translation matrix is applied to avoid negative coordinates.
- Finally, the first image is placed on top of the warped result.

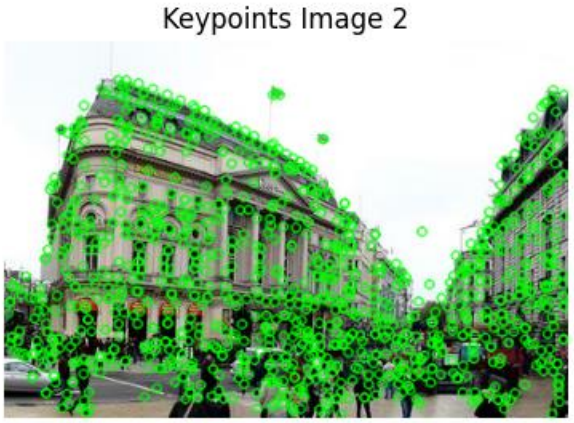
Results and Observations

Keypoints Detection

Both images produce a large number of stable keypoints, especially around:

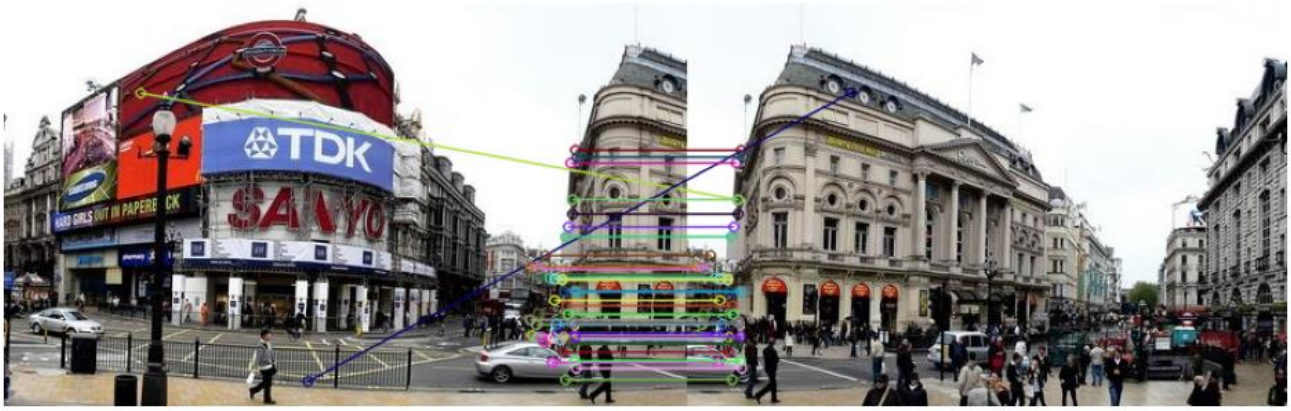
- Building edges
- Windows
- Strong texture regions

These points are essential for alignment.



Feature Matching

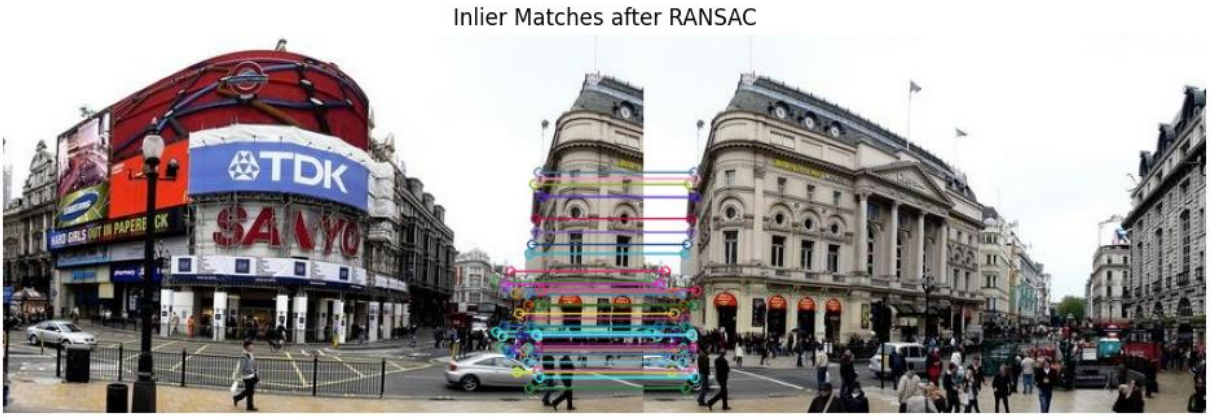
The ratio test significantly reduces incorrect matches, leaving only strong correspondences between overlapping regions.



RANSAC Inliers

After RANSAC:

- Outliers are removed
- Remaining matches align correctly along shared scene structure
- This improves geometric accuracy of stitching



Final Panorama Output

The final stitched image shows:

- A wide field-of-view panorama
- Correct alignment between overlapping regions
- Smooth global structure consistency

However, minor visible seam lines may still appear due to:

- Exposure differences
- Lack of blending between overlapping pixels

Overall Conclusion

The implemented panorama stitching pipeline successfully aligns and merges two overlapping images into a single wide-view panorama using classical computer vision techniques.

SIFT provides reliable feature detection, while RANSAC ensures robust geometric alignment. The final result demonstrates correct structure alignment but may still contain visible seam lines due to the absence of advanced blending techniques.

Overall, this task demonstrates a complete end-to-end feature-based image stitching system built entirely from classical methods without deep learning.

Task 6: Object Recognition and Localization

Objective

The goal of this task is to implement an automated object recognition and localization system. The pipeline identifies a specific reference object within a larger, more complex "query" scene by detecting shared features and calculating the geometric transformation required to map the object's location.

Methodology

The system uses a feature-based matching approach, leveraging the Scale-Invariant Feature Transform (SIFT) to handle variations in scale and orientation.

Pipeline:

1. **Auto-Cropping:** The reference image is automatically cropped using Canny edge detection and bounding boxes to ensure only the target object is used for feature extraction.

2. **Feature Extraction (SIFT):** Keypoints and descriptors are extracted from both the cropped reference and the query image.

3. **Feature Matching:** Keypoints are matched using a Fast Library for Approximate Nearest Neighbors (FLANN) matcher. Lowe’s ratio test (threshold = 0.75) is applied to filter out ambiguous matches.

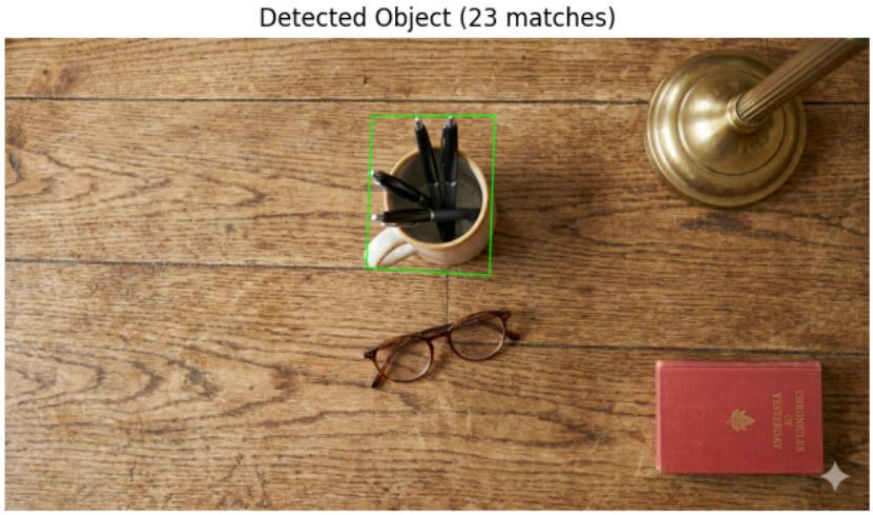
4. **Homography Estimation:** A perspective transformation matrix M is computed using the RANSAC algorithm to find the most mathematically consistent alignment while ignoring outliers.

5. **Localization:** The corners of the reference object are projected into the query image space using the matrix M, and a green bounding box is drawn to show the detection.

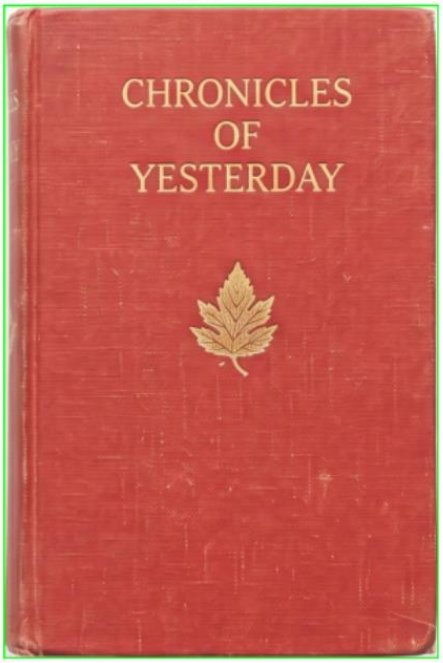
Strengths and Weaknesses

Stage	Strengths	Weaknesses
Auto-Cropping	Removes background noise from the reference, leading to cleaner descriptors.	May fail if the reference image has very low contrast or noisy edges.
SIFT + FLANN	Highly robust to rotation, scale, and minor lighting changes.	Computationally more expensive than simpler detectors like ORB.
RANSAC Homography	Effectively eliminates "false positive" matches that don't fit the geometric model.	Requires at least 4 high-quality matches to compute a valid transformation.

Results and Observations



Auto-Cropped Reference



Detected Object (100 matches)

The implementation successfully detects and localizes objects even when they are rotated or scaled differently in the query scene.

- **Accuracy:** By using **RANSAC**, the system maintains high localization accuracy even when the query image contains similar-looking background textures that might otherwise confuse the matcher.
- **Visual Feedback:** The output provides a side-by-side comparison, highlighting the cropped reference used and the final detection box in the scene, which clearly demonstrates the mathematical mapping.
- **Constraint:** If the object is heavily occluded (more than 70-80% hidden), the number of good matches may drop below the required threshold of 4, causing the homography calculation to fail.

Task 7: Depth Approximation from Two Images

Objective

The goal of this task is to estimate relative depth information from two images of the same scene captured from slightly different viewpoints.
The system analyzes how objects shift between the two images and uses this displacement to estimate which regions are closer or farther from the camera.

The final output is a depth map where:

- Brighter regions represent closer objects
- Darker regions represent farther objects

Methodology

Pipeline

Input Images → Grayscale Conversion → Block Matching → Disparity Computation → Normalization → Depth Visualization → Output

1. Stereo Image Selection

At the beginning of the implementation, regular `.png` stereo images were used.
However, the generated depth maps were noisy and inaccurate because the images were not perfectly rectified and did not contain ideal stereo alignment.

To improve the results, stereo images from the Middlebury Stereo Dataset were used instead with `.ppm` extension images.

The Middlebury dataset provides:

- High-quality stereo pairs
- Proper horizontal alignment
- Accurate disparity differences
- Standard benchmark stereo scenes

Using `.ppm` stereo pairs significantly improved depth estimation quality and reduced matching errors.

2. Grayscale Conversion

The RGB stereo images are converted to grayscale manually using a weighted intensity equation:

- Red channel contributes most strongly
- Green channel contributes moderately
- Blue channel contributes least

This simplifies the stereo matching process while preserving intensity structure.

Why grayscale?

Stereo matching depends on intensity similarity between regions rather than color information.

3. Block Matching

The core depth estimation algorithm is implemented manually using Block Matching.

For each pixel in the left image:

- A small window (block) around the pixel is extracted
- The algorithm searches horizontally in the right image

- The most similar block is selected

The search is limited to the same image row because stereo images are assumed to be horizontally aligned.

4. Similarity Measurement using SAD

To compare blocks, the Sum of Absolute Differences (SAD) method is used.

The algorithm:

- Computes pixel-wise absolute differences between blocks
- Sums all differences into a single matching cost

Lower SAD values indicate better matches.

Why SAD?

- Simple to implement
- Computationally efficient
- Commonly used in classical stereo vision

5. Disparity Computation

After finding the best matching block:

- The horizontal shift between corresponding pixels is computed
- This shift is called disparity

Relationship between disparity and depth:

- Large disparity → object is close
- Small disparity → object is far

The disparity values form the disparity map.

6. Disparity Normalization

The raw disparity values are normalized into the range [0,255].

This improves visualization by:

- Increasing contrast
- Making depth differences easier to observe
- Removing extreme outlier values

Percentile clipping is used to reduce noise effects.

7. Depth Visualization

A custom Jet color map was implemented from scratch for visualization.

Color interpretation:

- Red / Yellow → closer objects
- Blue / Dark → farther objects

This provides a clearer visual understanding of scene depth.

Challenges Faced During Implementation

1. Poor Results with PNG Images

Initially, normal PNG stereo images were used.
The resulting depth maps contained:

- Heavy noise
- Incorrect matching
- Weak object separation

This happened because the images were not ideal stereo pairs and lacked proper alignment.

Solution

The implementation was switched to Middlebury stereo images using `.ppm` format, which produced significantly better and more stable results.

2. Computational Cost

Manual block matching is computationally expensive because:

- Every pixel searches across multiple candidate blocks
- Nested loops are required

This increased runtime considerably for large images.

Solution

Image size and disparity search range were kept moderate to maintain acceptable runtime.

3. Incorrect Matches in Textureless Areas

Regions with little texture or repeated patterns caused matching ambiguity.

Examples:

- Flat walls
- Dark backgrounds
- Smooth surfaces

Effect

These regions occasionally produced noisy disparity values.

Results and Observations

Input Stereo Images

Two aligned stereo images from the Middlebury dataset were used as input.

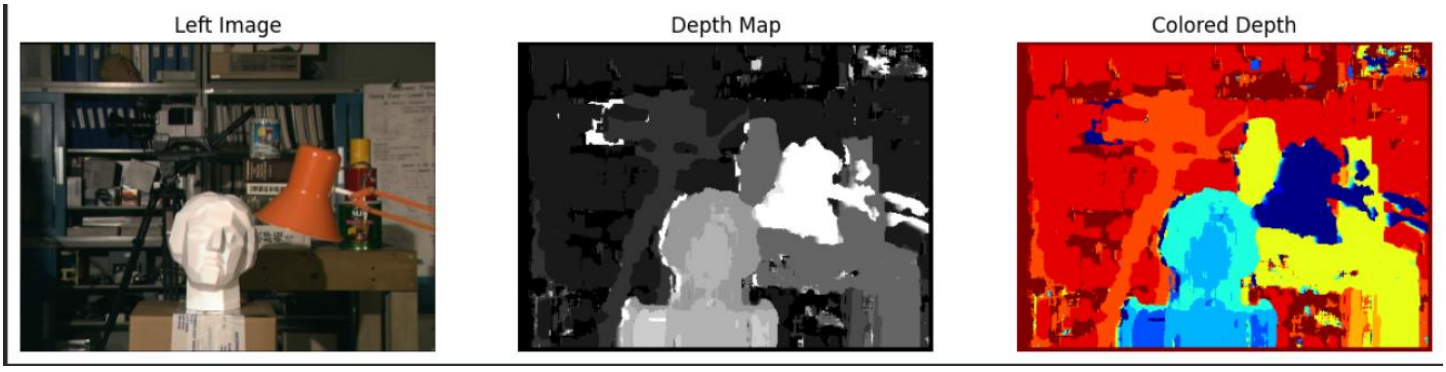
Output Depth Map

The generated depth map successfully distinguishes near and far objects.

Observations

- The statue in the foreground appears brighter because it is closer to the camera.
- Background shelves and distant objects appear darker.
- The colored depth map improves visual interpretation of depth variation.
- The Middlebury stereo pair produced significantly cleaner depth estimation compared to earlier PNG images.

Results



Strengths and Weaknesses

Component	Strengths	Weaknesses

Block Matching	Simple and interpretable	Computationally expensive
SAD Similarity	Fast and easy to implement	Sensitive to illumination changes
Manual Pipeline	Full understanding of stereo vision process	Slower than optimized OpenCV methods
Middlebury Stereo Images	High-quality and rectified	Limited to prepared stereo scenes

Overall Conclusion

A complete stereo depth estimation pipeline was successfully implemented from scratch using classical computer vision techniques.

The system performs:

- Manual grayscale conversion
- Block matching
- SAD similarity computation
- Disparity estimation
- Depth normalization
- Custom depth visualization

The use of Middlebury stereo images significantly improved the quality of depth estimation compared to ordinary PNG stereo images.

Task 8: High Dynamic Range (HDR) Imaging

What is HDR?

A standard camera can't capture very bright and very dark areas in one photo. HDR solves this by combining multiple photos taken at different exposures.

How It Works

The Problem

- **Short exposure** → Dark areas become black
- **Long exposure** → Bright areas become white
- One photo cannot show both

The Solution

Step 1: Take multiple exposures

- Very short (captures highlights)
- Normal (captures mid-tones)
- Very long (captures shadows)

Step 2: Align images
Camera moves between shots. Images are shifted to match exactly.

Step 3: Find camera response
Cameras don't record light linearly. We mathematically recover how the camera maps light to pixel values.

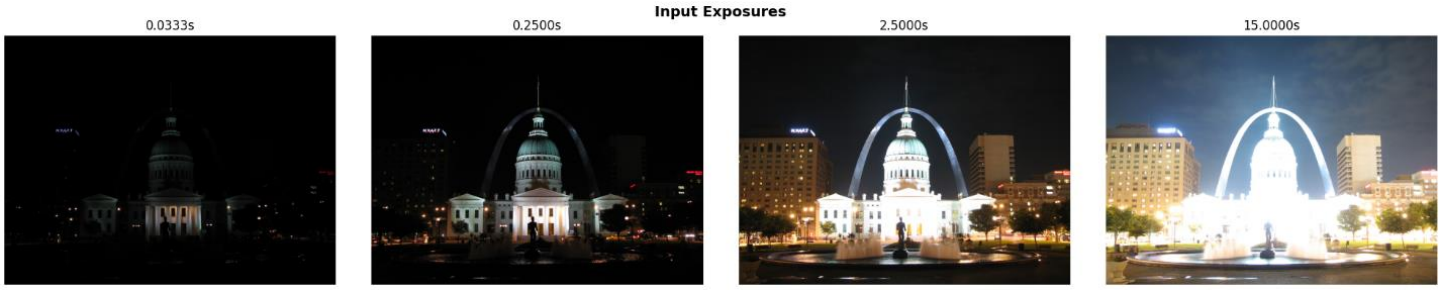
Step 4: Merge exposures
For each pixel:

- Short exposure → use for bright areas
- Long exposure → use for dark areas
- Combine with weights (mid-range pixels get highest weight)

Step 5: Tone mapping
HDR data is floating-point (0.001 to 100,000). Tone mapping compresses it to 0-255 for display while preserving visual information.

Results

Input:



Output:

Custom Pipeline

Why It Works

Human vision adapts to scene brightness. HDR mimics this by:

1. Measuring average scene brightness
2. Adjusting overall image to comfortable level
3. Preserving local contrast everywhere

Limitations

- Moving objects create "ghosts"
- Very long exposures add noise
- Requires multiple photos (scene must be static)

Conclusion

HDR combines multiple exposures to capture the full brightness range of real scenes, producing images that show details in both shadows and highlights—something no single photo can achieve.